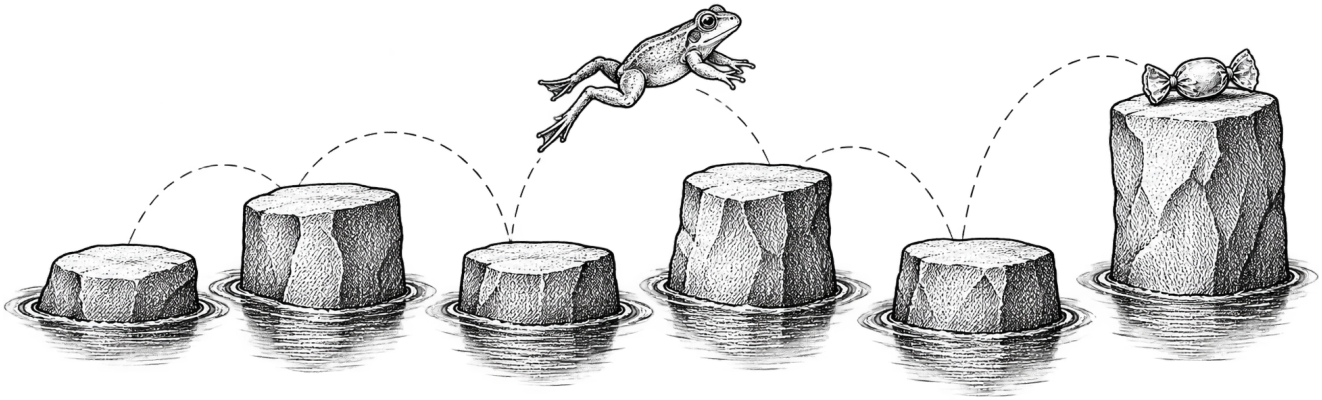
Staircase and Frog

Questions, diagrams and tables.

Try each question on paper or in the notebook.
Use the same question IDs to check your answers.



Understand what each step establishes
and why it leads to the next.

Open this lesson in Google Colab

fifteen02

JULIAN'S Pathfinder Foundation

Lesson 1 / Staircase and Frog

Warm-up: functions that call themselves

L1-01 / A function that calls itself

A function may call itself on a smaller input. countdown(n) below prints n and then calls countdown(n - 1); when n is 0 it prints Go! and makes no further call. In the table, write every call that countdown(3) makes, in order, what that call prints and which call it makes next.

```
def countdown(n):
    if n == 0:
        print('Go!')
        return
    print(n)
    countdown(n - 1)
```

Call	Prints	Calls next
countdown(3)		

L1-02 / Wait for the smaller answer

sum_squares(n) returns 0 when n is 0; otherwise it returns n * n + sum_squares(n - 1). A call cannot finish until the call it waits for has returned. Fill the table for sum_squares(4), starting from the bottom row and working up. Write the addition for each returned value.

```
def sum_squares(n):
    if n == 0:
        return 0
    return n * n + sum_squares(n - 1)
```

Call	Waits for	Returns
sum_squares(4)		
sum_squares(3)		
sum_squares(2)		
sum_squares(1)		
sum_squares(0)	nothing	0

L1-03 / Find the missing base case

power2(n) below should return 2 to the power n: 2 multiplied by itself n times, so power2(3) should be 8. What happens when you call power2(3)? Which value of n can you answer directly, without a smaller call? Add that base case and check that power2(3) now returns 8.

```
def power2(n):
    return 2 * power2(n - 1)
```

Start with:

```
def power2(n):
    return None # Replace with your answer.
```

What each function returns.

power2(n): the integer 2 raised to the power n.

Hints.

- Trace the inputs until they pass 0. Where should the chain stop?
- Check your repaired function on the stopping input before trying 3.

L1-04 / Write a recursive function

factorial(n) is the product 1 × 2 × ... × n, and factorial(0) is 1. Write factorial(n) recursively: first the base case, then one smaller call. Predict factorial(4) before running it. The staircase that follows uses the same pattern, with two smaller calls instead of one.

Start with:

```
def factorial(n):
    return None # Replace with your answer.
```

What each function returns.

factorial(n): the product of integers 1 through n; for n = 0, return 1.

Hints.

- What is the smallest n you can answer directly?
- If you already knew factorial(n - 1), what single step would give factorial(n)?

Staircase: count, remember, then loop

L1-05 / Five steps, many routes

Start on the ground, called step 0. Each move climbs exactly 1 or 2 steps. Count routes that land exactly on each target from step 1 to step 5. List the routes to step 5 as move lists, such as [1, 2, 2]. Does changing the order make a different route?

Step	1	2	3	4	5
Routes					

L1-06 / The last move explains the sum

The next move must land on step 5. Where can that move start? Use your L1-05 counts for step 4 and step 3. How can these two counts give the answer for step 5?

L1-07 / Where recursion stops

Let ways(n) return the number of routes from the ground to target step n, using jumps of 1 or 2. What should ways(0) and ways(-1) return? Use these answers to work out ways(1).

L1-08 / The slow recursive program

Write stairs_slow(n), the number of staircase routes to step n with jumps of 1 or 2. Start with the stopping cases, then ask about the two possible starting steps of the final jump. Draw every call made for stairs_slow(3) and write the returned count beside it.

Start with:

```
def stairs_slow(n):
    return None # Replace with your answer.
```

What each function returns.

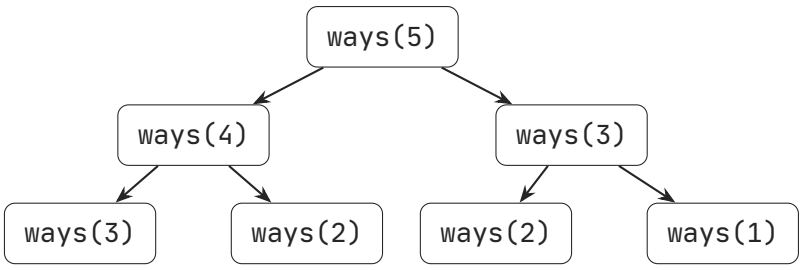
stairs_slow(n): the number of ordered routes to step n using jumps of 1 or 2.

Hints.

- Which targets can you answer without a smaller call? Think about 0 and about negative targets.
- The last move is 1 or 2 steps. Which smaller targets do those leave?

L1-09 / Repeated questions in the call tree

The diagram shows the first three levels of calls made while counting routes to step 5. Circle calls with the same input. What work does the second call to ways(3) repeat?



L1-10 / A cache keeps an answer

A cache is a dictionary that saves an answer under the input that produced it. We have found 5 routes to step 4. Complete cache[?] = ?. What should a later call to ways(4) return, and which work can it skip?

L1-11 / Fast work can still mean deep recursion

Caching removes repeated work. Why can a large staircase input still cause RecursionError in Python? What does iteration change?

L1-12 / Build the table in order

Write stairs_table(n) using a loop. First fill ways[step], the route count, for steps 0 through 5. Which earlier count is available at step 1? Which two are needed from step 2 onwards? Use that order in your code.

Start with:

```
def stairs_table(n):
    return None # Replace with your answer.
```

What each function returns.

stairs_table(n): the number of ordered routes to step n using jumps of 1 or 2.

Hints.

- How long must the list be so that ways[n] exists?
- Which entry do you know before the loop starts?
- When you fill ways[step], which entries must already be filled?

Step	0	1	2	3	4	5
ways[step]	1	?	?	?	?	?

L1-13 / Your turn: jumps of 1 or 3

Write stairs_13(n) by adapting stairs_table from L1-12. Allow only jumps of 1 or 3, never 2. First fill the counts for steps 0 through 7, then write and test your code.

Start with:

```
def stairs_13(n):
    return None # Replace with your answer.
```

What each function returns.

stairs_13(n): the number of ordered routes to step n using jumps of 1 or 3.

Hints.

- Which line of stairs_table depends on the allowed jumps?
- A jump can only start from a step that exists. What must be true before you read an earlier entry?

Step	0	1	2	3	4	5	6	7
ways[step]	1							

Frog: compare complete route costs

L1-14 / Frog: cost belongs to a whole route

Number the stones 0, 1 and 2; their heights are [10, 20, 30]. A frog starts at stone 0 and may jump forward one or two positions. A jump costs the absolute height difference, so a drop of 10 also costs 10. List all routes to stone 2 and add the jump costs for each.

L1-15 / A greedy choice can fail

Use heights [20, 50, 30, 40, 10, 10], at stones 0 through 5. At each stone, choose the cheaper of the next two legal jumps; if costs tie, choose the farther stone. Trace this greedy route and its total cost. Can another complete route cost less?

Stone i	0	1	2	3	4	5
heights[i]	20	50	30	40	10	10

L1-16 / Frog: ask about the last jump

Write frog_slow(heights) with recursive helper cost(i): the cheapest total cost from stone 0 to stone i. Which earlier stones can supply the last jump? Explore both possibilities.

Start with:

```
def frog_slow(heights):
    def cost(i):
        return None # Your helper goes here.
    return cost(len(heights) - 1)
```

What each function returns.

frog_slow(heights): the cheapest total cost from stone 0 to the last stone.
cost(i): the cheapest total cost from stone 0 to stone i, including all jumps along the route.

Hints.

- What is the smallest i you can answer directly?
- Which earlier stones can the last jump into stone i start from?
- What does cost(i) need to know about each of those stones?

L1-17 / Frog: cache the stone number

In the recursive Frog solution, what is the cache key? Why can different routes that reach the same stone share an answer?

L1-18 / Frog: write the iterative solution

Write frog_table(heights) using a loop. Return the cheapest total cost from stone 0 to the last stone. First fill best[i] for heights [20, 50, 30, 40, 10, 10], then turn those steps into Python code. Explain why the loop runs from left to right. Test your function on these heights.

Start with:

```
def frog_table(heights):
    return None # Replace with your answer.
```

What each function returns.

frog_table(heights): the cheapest total cost from stone 0 to the last stone.

Hints.

- What is best[0], before any jump?
- Which earlier entries does best[i] need, and are they filled when the loop reaches i?

Stone i	0	1	2	3	4	5
heights[i]	20	50	30	40	10	10
best[i]	0	?	?	?	?	?

L1-19 / Your turn: a new Frog instance

Test your frog_table(heights) from L1-18. For heights [10, 30, 20, 40], first predict best[i] and give one cheapest route, then run your function. Next predict and test the answer for 100,000 equal-height stones. Explain why uncached recursion is unsuitable for that large test.

Stone i	0	1	2	3
heights[i]	10	30	20	40
best[i]	0			

L1-20 / Counting and minimising

Both staircase and Frog inspect earlier states. Why does staircase add their answers, while Frog takes a minimum? What does each table entry mean?